

Universidad de San Andrés, Buenos Aires, Argentina



I206 - Inferencia y Estimación

Trabajo Práctico II

Teoría de la Información:

Codificación de Huffman

Camiscia, Luca

lcamiscia@udesa.edu.ar

Legajo: 36781

Palavecino, Axel

apalavecino@udesa.edu.ar

Legajo: 36778

Pereyra, Agustín

apereyra@udesa.edu.ar

Legajo: 36631

30 de octubre de 2025

Abstract

Este trabajo investiga la relación fundamental entre la *distribución probabilística* de los datos y su límite teórico de compresibilidad, la **entropía**. Mediante la utilización del algoritmo de **Codificación de Huffman** en textos con distribución de caracteres distintas, como lenguaje natural, tabla de datos o secuencias de ADN. Se cuantifica experimentalmente su eficiencia y se compara con algoritmos de compresión uniforme. Los resultados demuestran que el rendimiento de la compresión está directamente determinado por la redundancia de la fuente, logrando una reducción de memoria de hasta un 41.4 % en textos con baja entropía, mientras que es nula en datos con distribución uniforme. Esto confirma que la codificación de Huffman genera una longitud promedio de código que se aproxima al límite establecido por la entropía, destacando un principio fundamental de la *Teoría de la Información*.

1. Introducción

Vivimos en un mundo donde la información crece exponencialmente, en la era del *Big Data* cada día se generan millones de textos, imágenes, videos y datos de todo tipo. En este contexto, nos surge un desafío: *¿Cómo podemos almacenar y transmitir toda esta información de manera eficiente y compacta?* En trabajos previos hemos abordado este problema a partir de la reducción dimensional mediante técnicas como PCA, metodo que permite representar datos complejos, como matrices, en espacios de menor dimension, y así compactar su peso en memoria sin gran pérdida en representatividad. Ahora, al tratar con textos, el desafío es similar pero más estricto, necesitamos **comprimir** la información sin perder ni un solo carácter.

Este problema fue abordado, hace 70 años, por **Claude Shannon**, quien interpretó la **Teoría de la Información** y brindó un límite teórico fundamental para la compresión de datos. Este límite está directamente relacionado con la **entropía** de la fuente de datos, que representa una medida de su incertidumbre o "sorpresa" promedio. Para una fuente $\mathbf{X}$ con N símbolos únicos de probabilidad $p(x_i)$, la entropía se define como:

$$H(\mathbf{X}) = - \sum_{i=1}^N p(x_i) \log_2(p(x_i)) \quad (1)$$

En esencia, la entropía $H(\mathbf{X})$ indica la cantidad mínima de bits necesarios, en promedio, para codificar cada símbolo de la fuente sin perder información. La demostración de Shannon establece que ningún algoritmo de compresión puede lograrlo con menos de $H(\mathbf{X})$ bits por símbolo en promedio.

Esta limitación teórica tiene implicaciones prácticas en, por ejemplo, textos de lenguaje natural. Consideremos un texto en español, donde ciertas letras como "e" o "a" aparecen con mucha más frecuencia que otras como "q" o "z", o ciertos símbolos como "\$" o "%". Esta *distribución no uniforme* de los símbolos implica que el texto tiene **redundancia**, es decir, gran parte de su contenido es repetitivo y **predecible**.

Precisamente en esta redundancia se basa la **Codificación de Huffman**, un método que aprovecha las frecuencias de cada símbolo para asignar códigos más cortos a símbolos comunes. El desempeño de este método se mide a través de la **longitud promedio de codificación**, $L(\mathbf{X})$, que es el costo esperado en bits por símbolo, calculado como:

$$L(\mathbf{X}) = \sum_{i=1}^N p(x_i) \ell(x_i) \quad (2)$$

donde $\ell(x_i)$ es la longitud del código para el símbolo x_i . Esta longitud promedio siempre estará acotada por la entropía, lo que nos permite evaluar su cercanía al óptimo teórico:

$$H(\mathbf{X}) \leq L(\mathbf{X}) < H(\mathbf{X}) + 1 \quad (3)$$

Para cuantificar el valor exacto que estamos ganando al utilizar este método, comparamos $L(\mathbf{X})$ con una codificación de referencia que no siempre es óptima (únicamente óptima en el caso que se maximice la entropía), como la *Codificación Uniforme*. Esta codificación asigna a cada símbolo un código de la misma longitud fija, conocida como **longitud uniforme mínima**, y se calcula como:

$$L_{\text{unif}}(\mathbf{X}) = \lceil \log_2(N) \rceil \quad (4)$$

La efectividad, o eficiencia, del algoritmo de Huffman se puede medir entonces como el porcentaje de **reducción de memoria**:

$$R(\mathbf{X}) = \left(1 - \frac{L(\mathbf{X})}{L_{\text{unif}}(\mathbf{X})} \right) \times 100 \% \quad (5)$$

Un problema que surge es que necesito comparar cadenas de textos con distintos tamaños de alfabetos Σ . Para solucionar este problema definimos una medida de incertidumbre normalizada, la **entropía normalizada de Shannon**, la cual se define como:

$$\eta(\mathbf{X}) = \frac{H(\mathbf{X})}{\log_2(N)} \quad (6)$$

Este trabajo busca validar experimentalmente estos principios. Nuestra hipótesis es que la efectividad del algoritmo de Huffman, medida por $R(\mathbf{X})$, es directamente proporcional a la redundancia de la fuente. Principalmente esperamos que sea significativamente más efectivo en textos de lenguaje na-

tural que en secuencias con distribuciones más uniformes, donde la entropía se maximiza. A través de este análisis, ilustraremos de forma práctica cómo la estructura probabilística de los datos determina no solo *si* podemos comprimirlos, sino también *cuánto*.

2. Desarrollo Experimental

Nuestra metodología se dividió en tres pasos principales. Primero, estimamos el modelo probabilístico de cada fuente, basándonos en las frecuencias de sus símbolos. Luego, aplicamos el algoritmo de Huffman que nos proporcionó la Cátedra para generar los códigos binarios necesarios. Finalmente, hicimos un análisis detallado usando las métricas de desempeño que definimos antes, con el objetivo de comparar los distintos textos.

2.1. Fuentes de Datos

Para probar nuestra hipótesis de cómo la distribución de símbolos influye en la compresibilidad, la Cátedra nos dio tres textos con características muy distintas. El primero es una secuencia de ADN (`adn.txt`), que es el caso más sencillo: un alfabeto de solo 4 símbolos (adenina, citosina, guanina y timina, representados como ‘a’, ‘c’, ‘g’ y ‘t’).

En el otro extremo, tenemos un texto de lenguaje natural, un fragmento sobre mitología en *español* (`mitologia.txt`). Aquí el alfabeto crece para incluir las 27 letras del español, números, signos de puntuación y espacios. A diferencia del ADN, el lenguaje natural no tiene una distribución uniforme, sino que sigue patrones lingüísticos que todos conocemos.

Finalmente, hay un caso intermedio que no es tan obvio, se trata de un archivo de datos estructurados en formato JSON (como `tabla.txt`). Este archivo mezcla texto, números decimales y símbolos especiales del formato (como { , } , : , , , etc.). A diferencia de un texto narrativo, su estructura sigue reglas muy estrictas de formato, lo que crea una distribución de símbolos distinta tanto del ADN como del lenguaje natural. No es fácil predecir cuánto se puede comprimir sin hacer el análisis.

Cada una de estas fuentes se procesó siguiendo la metodología que describimos a continuación, lo que nos permitió comparar cómo la naturaleza de la fuente influye en su límite de compresión.

2.2. Modelo probabilístico de la Fuente

El objetivo en esta etapa es estimar una **función de masa de probabilidad (PMF)** de cada fuente

de texto. El resultado de este paso es un conjunto de pares (s_i, p_i) que representan los símbolos y sus probabilidades de ocurrencia, es decir, su **probabilidad empírica** de aparición $p(x)$. Este diccionario constituye el modelo estadístico de la fuente, y es el insumo fundamental sobre el cual operará el algoritmo de **Huffman**.

En general, la probabilidad estimada del i -ésimo símbolo s_i con su frecuencia asociada f_i se calcula como:

$$p(x_i) = \frac{f_i}{M} \quad (7)$$

donde cada caracter es una de las N realizaciones totales de la variable aleatoria fuente S y $M = \sum_{j=1}^N f_j$, en otras palabras la longitud total de la fuente.

La utilidad y desarrollo de esta etapa pueden variar dependiendo del objetivo de la compresión. Por ejemplo, si el objetivo es maximizar la compresión de una novela, podría optarse por no contar las mayúsculas, ya que no afectan significativamente la estructura del texto. En contraste, en lenguajes de programación donde las mayúsculas son relevantes (como en identificadores de una variable global vs. local), este enfoque no sería apropiado. En este informe se tratarán diversas configuraciones de mayúsculas y minúsculas para cuantificar la compresibilidad en cada caso.

2.3. Implementación del Algoritmo de Huffman

Luego de obtener el modelo probabilístico de la fuente, generamos un conjunto de códigos binarios utilizando el algoritmo de **Codificación de Huffman**. La característica que diferencia este código de otros universales como el *código Morse* es que ninguna cadena de código asignada a algún símbolo es prefijo de otra. A esta propiedad se la conoce como *prefix-free*, y es gracias a esto que su decodificación es inmediata.

Para lograr esto, el código se construye como un árbol binario, cuyas hojas representan un símbolo y a cada rama un elemento binario. Los caracteres de

menor frecuencia se encuentran en los mayores niveles de profundidad del árbol, lo que tiene sentido porque los más frecuentes serán los más rápidos de decodificar, y esto es justamente lo que buscamos.

El árbol se genera de la forma representada en el Algoritmo 1: se añaden los símbolos de menor a mayor frecuencia, creando subárboles de manera que la frecuencia del nodo padre es la suma de la frecuencia de sus hijos. De esta manera, iterativamente se van uniendo los subárboles, dejando una estructura como la de la Figura 1.

Algoritmo 1 HuffmanTree

Input: pmf $p(s)$ de los caracteres s_i de la cadena S
 $Q \leftarrow$ priority-queue de nodos $n_i = \langle s_i, p(s_i) \rangle$
 # orden ascendente de p
while $\text{len}(Q) > 1$ **do**
 $n_1, p_1 \leftarrow Q.\text{pop}()$
 $n_2, p_2 \leftarrow Q.\text{pop}()$
 $n_{\text{padre}} \leftarrow \langle \text{sub-árbol}(n_1, n_2), p_1 + p_2 \rangle$
 $Q.\text{enqueue}(n_{\text{padre}})$
end while
 $\text{root} \leftarrow Q.\text{pop}()$
return root

Luego la codificación se logra asignando a cada rama un valor binario y para cada símbolo se toma los valores de las ramas del camino único que se realiza desde la raíz hasta la hoja que representa a ese símbolo. En el ejemplo de la figura la codificación para una cadena “abcdabcaba” quedaría tal que $a = 0$, $b = 10$, $c = 110$ y finalmente $d = 111$.

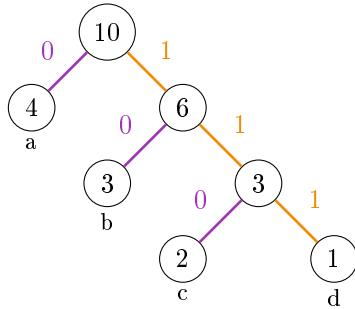


Figura 1: Ejemplo de un Árbol de Huffman para la cadena $S = \text{“abcdabcaba”}$

2.4. Medidas de Desempeño y Análisis Cuantitativo

Una vez que tenemos la codificación de Huffman generada para cada cadena $\mathbf{X}$ de N caracteres x_i únicos, con su respectiva distribución de frecuencias, necesitamos métricas concretas que nos permitan evaluar su efectividad. En la Introducción definimos formalmente cada una de estas medidas; ahora explicaremos cómo interpretarlas en el contexto práctico de nuestro análisis.

Longitud Promedio de Codificación

La longitud promedio $L(\mathbf{X})$ (Ecuación (2)) es definida como la esperanza de la longitud del código de la cadena y su cálculo es simplemente el costo promedio en bits por símbolo. Es la métrica central que nos indica qué tan compacta es nuestra codificación.

Con este podemos darnos una idea de qué tan complejo es la codificación que nos aporta el algoritmo de Huffman. A mayor L , podemos inferir 2 posibles casos, que haya una gran cantidad de caracteres y por lo tanto el árbol binario es de una mayor profundidad o simplemente que la codificación sea ineficiente en términos de compresión.

Longitud Uniforme Mínima

La longitud uniforme $L_{\text{unif}}(\mathbf{X})$ (Ecuación (4)) expresa la mínima cantidad de bits por símbolo necesaria para una codificación uniforme, es decir que no tiene en cuenta la frecuencia de aparición de cada uno de los símbolos.

Es muy útil a la hora de evaluar la efectividad de la codificación de Huffman de manera que si $L < L_{\text{unif}}$, ciertamente el algoritmo explota la redundancia de la cadena.

Reducción de Memoria

El porcentaje de reducción $R(\mathbf{X})$ (Ecuación (5)) es el porcentaje relativo de ahorro de memoria que aporta la codificación se puede estimar comparando L con L_{unif} .

Notar que, un valor cercano a 0 % significa que la codificación de Huffman no aporta ventajas significativas sobre el método uniforme, lo que sucede cuando el texto tiene una distribución muy uniforme de caracteres. Por el contrario, valores altos (cercaos a 100 %) indican una compresión muy efectiva, típica de textos con alta redundancia. Es importante notar que si $L \geq L_{\text{unif}}$, entonces $R \leq 0 \%$, lo que confirmaría que Huffman no es apropiado para esa fuente. Esto es, en gran parte, los fundamentos de la hipótesis que buscamos validar.

Entropía de Shannon

La entropía $H(\mathbf{X})$ (Ecuación (1)) nos da el límite teórico absoluto, es decir, la cantidad mínima de bits por símbolo que cualquier algoritmo de compresión sin pérdida podría alcanzar para este texto.

Una forma que aporta gran valor a nuestro análisis es la interpretación de este valor usando su relación con la longitud promedio de codificación, dada por la Ecuación (3). Esta desigualdad nos permite evaluar la eficiencia de la codificación de Huffman, dándonos un límite inferior para L . En caso de que

sean iguales, la codificación tiene una eficiencia máxima, y en el caso contrario en que $L > H$ se confirma que existe cierta redundancia en la cadena.

Entropía Normalizada

El problema de la medida anterior es que se encuentra escalada respecto a la cantidad de símbolos

únicos (N) de la cadena, es por eso que a la hora de realizar comparaciones entre cadenas de diferentes N es necesario utilizar la versión normalizada (Ecuación (6)), donde $\eta \in [0, 1]$ tal que en sus valores extremos $\eta = 1$ evidencia una incertidumbre total o una distribución totalmente uniforme de los símbolos, mientras que en $\eta = 0$ la cadena sería una repetición continua de un solo caracter.

3. Análisis de Datos

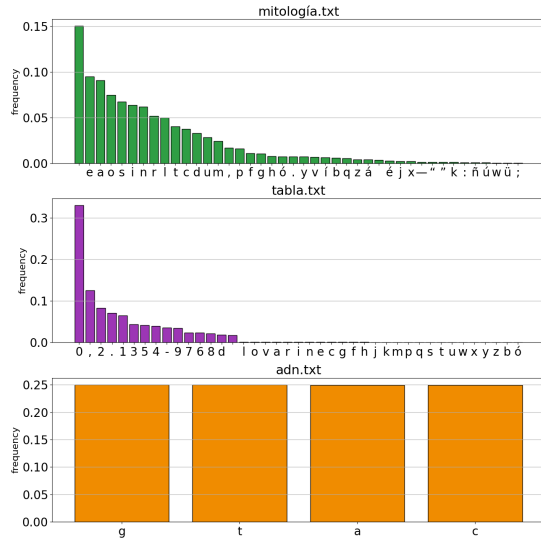


Figura 2: Función de masa de probabilidad de los caracteres de cada uno de los textos de ejemplo

En un primer vistazo al gráfico, notamos que el archivo *adn.txt* muestra una distribución completamente uniforme entre los cuatro símbolos que lo componen, lo que significa que $L = L_{\text{unif}}$. Cada uno de estos aparecen con una frecuencia muy cercana al 25 %.

Desde el punto de vista de la teoría de la información, una fuente con probabilidades equiprobables no contiene redundancia. En consecuencia, la codificación de Huffman no logra ninguna compresión resultando en una reducción relativa de memoria nula, como se muestra en el cuadro 1. La entropía normalizada corrobora lo mencionado ob-

servando que para este caso, con una precisión de 4 decimales es igual a 1. Como mencionamos en la sección anterior, gracias a este dato inferimos que la cadena tiene una distribución (casi) perfectamente uniforme de caracteres.

Por otro lado, el archivo *mitología.txt* muestra una distribución de frecuencias mucho más variada. El texto cuenta con 42 símbolos distintos, cuya entropía de 4.27 bits/símbolo se traduce en una longitud promedio muy cercana a dicho valor. Se observa cómo unas pocas letras concentran una gran cantidad de apariciones, lo cual es característico del lenguaje natural. El carácter más utilizado resulta ser el “ ” (espacio), en concordancia con la regla gramatical que establece su uso para separar palabras. Esta redundancia permite al algoritmo de Huffman asignar códigos más cortos a los símbolos de mayor frecuencia. Gracias a ello, se obtiene una longitud promedio de 4.27 bits/símbolo y una reducción de memoria del 28.31 %. En este caso, puede apreciarse claramente cómo la redundancia inherente al lenguaje natural puede aprovecharse para comprimir información de manera eficiente. Su entropía normalizada ayuda a confirmar esta observación, ya que al situarse debajo del valor máximo, evidencia que el texto no es completamente aleatorio sino que presenta patrones estadísticos predecibles en la aparición de ciertos caracteres. Estos patrones (como la alta frecuencia de vocales, espacios y algunas letras) introducen una estructura que reduce la incertidumbre promedio por símbolo. Podemos afirmar que su menor entropía es precisamente lo que hace posible una compresión efectiva.

	Entropía	Entropía Normalizada	Símbolos Únicos	Longitud Promedio	Longitud Uniforme Mínima	Reducción de Memoria
	H [bits/símbolo]	$\eta \in [0, 1]$	N [cantidad]	L [bits/símbolo]	L_{unif} [bits/símbolo]	R [%]
adn.txt	1.9999	1.0000	4	2.0000	2	0.0000
mitología.txt	4.2685	0.7916	42	4.3013	6	28.3112
tabla.txt	3.4668	0.6471	41	3.5161	6	41.3978

Cuadro 1: Resumen de los cálculos de las medidas mencionadas en la sección anterior realizados para los textos de ejemplo

Finalmente, el archivo `tabla.txt`, al corresponder a una tabla de valores, presenta una distribución dominada por caracteres numéricos y signos de puntuación. A diferencia del texto anterior, esta distribución no obedece patrones lingüísticos, sino una organización propia de los datos estructurados. En consecuencia, unos pocos símbolos concentran la mayor parte de las apariciones, generando una fuerte desigualdad en las probabilidades y por lo tanto una entropía considerablemente menor (Observable al comparar entropías normalizadas). Este comportamiento se traduce en la mayor compresibilidad

de los tres casos analizados, con una reducción de memoria del 41.30 %. La longitud promedio de 3.46 bits/símbolo evidencia una fuente altamente redundante desde el punto de vista estadístico. En otras palabras, el algoritmo de Huffman puede asignar códigos especialmente cortos a los caracteres dominantes y alcanzar así la máxima eficiencia de codificación observada.

En conjunto, estas diferencias exhiben como las distribuciones de frecuencia están fuertemente determinados por la naturaleza de la información influyendo directamente en su compresibilidad.

4. Conclusión

Del análisis pudimos observar que la naturaleza del contenido tiene una fuerte correlación con su compresibilidad. También podemos observar como el conocer en profundidad el texto a codificar permite realizar modificaciones al algoritmo para una mayor ventaja. Aun teniendo en cuenta que cuando queremos aplicar Huffman **no se debe perder absolutamente nada de información**, esto no significa necesariamente que el mensaje reconstruido sea el mismo. El mejor ejemplo es el aplicado en este informe de tomar la decisión de no diferenciar mayúsculas de minúsculas. En un texto natural es posible identificarlas con un programa simple que las reemplace luego de signos de puntuación o para sustantivos propios. Notar que esto solo es aplicable para textos de lenguaje natural ya que en otro caso toda las mayúsculas podrían ser una componente clave que cambie el significado del mensaje. Así como tomamos a una minúscula y una mayúscula como fuente de la misma información, otra consideración que podría surgir es ser tomar a cualquiera de las variantes de cada carácter con cualquiera sea el tipo de diacrítico que posee como una misma. Por ejemplo diríamos que $a \equiv \{a, \acute{a}, \grave{a}, \hat{a}, \ddot{a}, \dots\}$ (equivalentes en términos de información). E ignorar caracteres de puntuación como $\{“, ”, “, “, \dots\}$. En muchos casos la ausencia de una acentuación puede dificultar la compresión pero no la imposibilita, y usualmente con el contexto se puede decifrar fácilmente la intención transmitida. Otro caso es el de la puntuación, en donde una sola coma puede

cambiar totalmente esa intención por lo que no es posible realizar este arreglo.

Con el objetivo de una comparación extremista, realizamos los mismos cálculos para las versiones “restringidas” de los textos en los que la puntuación es ignorada y los caracteres con diacríticos son equivalentes al carácter sin este. En la Figura 3 se encuentra la representación gráfica de la PMF de esta versión.

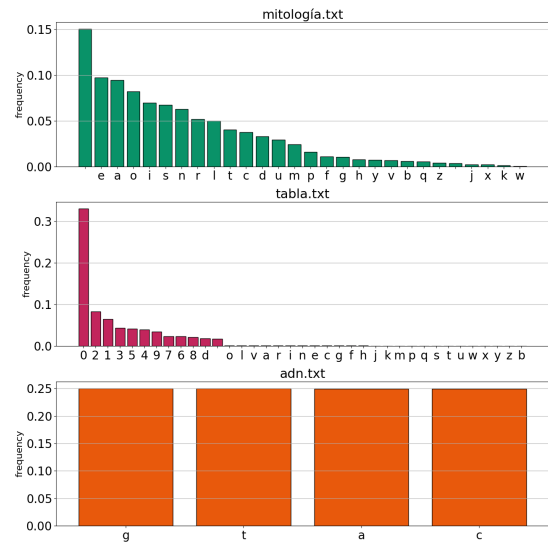


Figura 3: Función de masa de probabilidad de los caracteres de cada uno de los textos de ejemplo (versión “restringida”)

	Entropía	Entropía Normalizada	Símbolos Únicos	Longitud Promedio	Longitud Uniforme Mínima	Reducción de Memoria
	H [bits/símbolo]	$\eta \in [0, 1]$	N [cantidad]	L [bits/símbolo]	L_{unif} [bits/símbolo]	R [%]
adn.txt	1.999999	1.000000	4	2.000000	2	0.000000
mitología.txt	3.517115	0.748252	26	3.302169	5	33.956622
tabla.txt	2.546965	0.492650	36	2.265255	6	62.245757

Cuadro 2: Resumen de los cálculos de las medidas para los textos de ejemplo (versión “restringida”)

Nuestro experimento con versiones “restringidas” de los textos muestra que eliminar ciertos caracteres puede tener efectos adversos incluso en los casos donde parecería inofensivo. En `mitología.txt`, la supresión de signos como las comas y los puntos altera la estructura del lenguaje y modifica el sentido de las oraciones, por lo que la cadena resultante deja de representar fielmente el texto original. De forma similar, en `tabla.txt` la eliminación de estos mismos caracteres borra la referencia decimal y distorsiona el valor de los números, haciendo que los datos pierdan precisión. Este resultado evidencia como un carácter que parece “redundante” en un cierto contexto puede ser esencial para conservar la precisión en un texto de otro tipo.

En resumen, la compresibilidad no es una pro-

piedad intrínseca de un algoritmo sino una interacción entre éste y la estructura de los datos. Las fuentes uniformes son incomprensibles, los textos naturales contienen cierta redundancia explotable y los datos altamente estructurados pueden compactarse de manera notable. Además, pequeñas transformaciones en la representación —como unificar letras mayúsculas y minúsculas o ignorar acentos— permiten aumentar la eficiencia cuando se conoce el dominio de aplicación y se garantiza que el significado no se ve comprometido. Estas consideraciones subrayan la importancia de combinar técnicas de teoría de la información con conocimiento contextual para desarrollar métodos de compresión realmente adaptados a cada tipo de fuente.